Why attention divides by $\sqrt{d}$

Attention lets each word decide how much to listen to every other word. When the vectors behind it get long, those decisions turn all-or-nothing. Dividing by $\sqrt{d_k}$ keeps them soft.

Based on Attention Is All You Need, Vaswani et al., 2017

1 From scores to weights

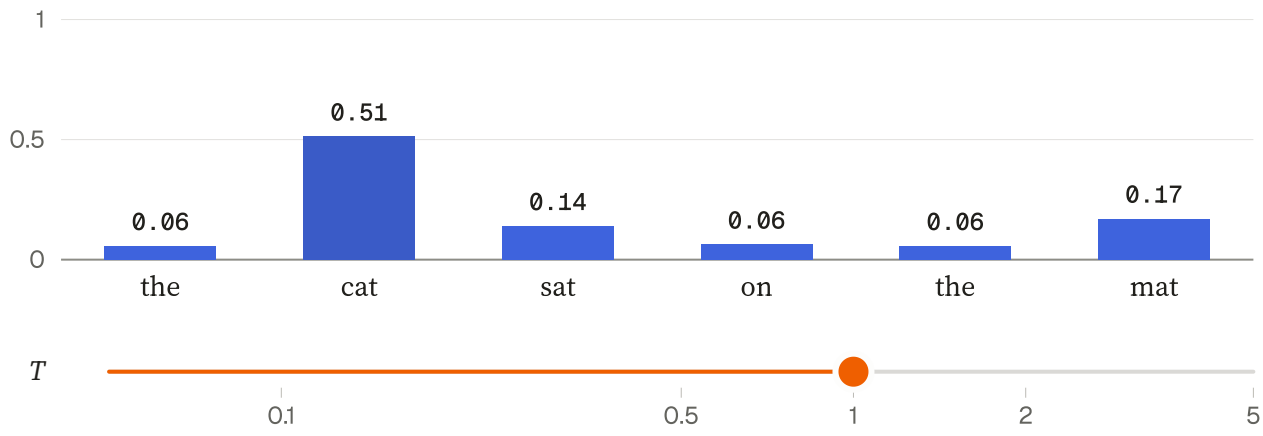
A transformer reads a sentence by letting every word ask a question of every other word. The asking word holds a *query* vector q, each word it looks at offers a *key* vector k, and their dot product $q \cdot k$ is a score: how relevant that word looks. Scores can be any real number, so attention turns them into weights with a softmax. The weights are positive and add up to 1.

$$w_i = \exp(s_i / T) \,/\, \Sigma_j \exp(s_j / T)$$

Each weight w is its score s, divided by the temperature T and exponentiated, as a share of the total.

Below, the word “sat” is asking about the six words of “the cat sat on the mat”. The scores are fixed by hand so you can watch the softmax on its own. The one knob is the temperature T, which divides every score before the softmax. Drag it.

Temperature sets how sharp the softmax is



top weight **0.51** entropy **2.05** bits

Shown at $T = 1$. Interactive version at explorable.geekplux.com/s/attention#fig-temperature

The scores are fixed by hand; the softmax is computed live. Drag the temperature T and watch how the word “sat” spreads its attention: one word takes over or the weights even out.

2 Sharp or soft is a matter of scale

At $T = 1$, “cat” gets about half of the attention and the rest is shared. Drag T down to 0.25 and “cat” takes 98%: the entropy, a measure of how spread out the weights are, falls from 2.05 bits to 0.15. Push T up to 5 and the weights flatten toward 1/6 each, close to the 2.58-bit maximum for six words. The order of the scores never changes. Only their scale does, and the scale alone decides whether attention is a spotlight or a floodlight.

The spotlight end is a problem for learning. How much a weight moves when you nudge a score depends on the weights themselves. When one weight is nearly 1 and the rest are nearly 0, a nudge to any score barely moves anything, so a training signal that tries to move attention elsewhere barely registers. A saturated softmax has stopped listening.

$$\partial w_i / \partial s_j = w_i(\delta_{ij} - w_j)$$

The change is a product of weights. If one weight is 1 and the others 0, every product is 0.

3 Where the scale comes from

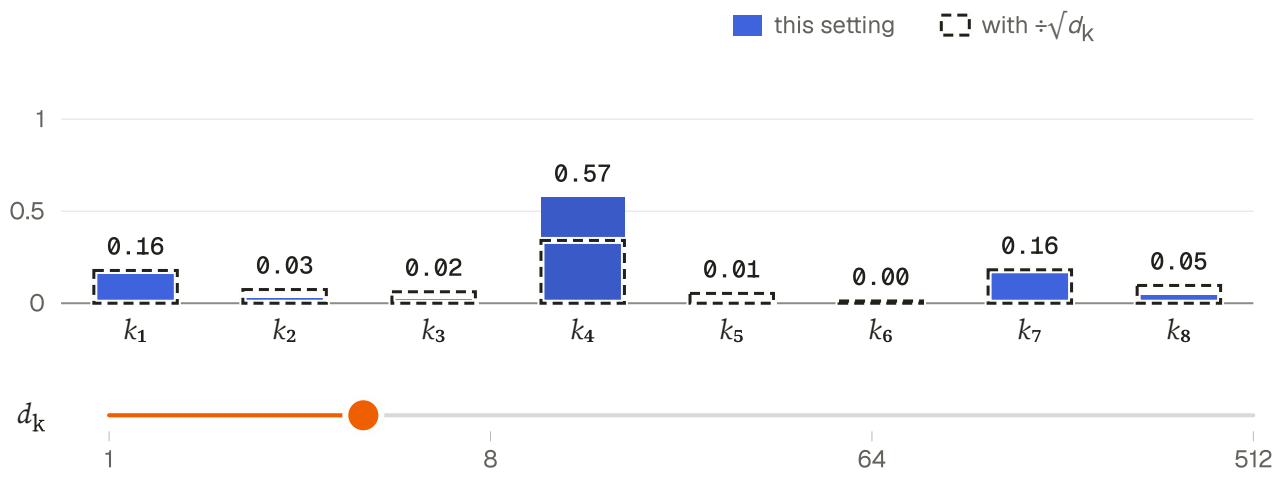
Nobody sets a temperature by hand inside a transformer. The scale of the scores comes from the vectors themselves. A dot product of two d_k-dimensional vectors is a sum of d_k products, one for each pair of matching components. If every component is random with mean 0 and variance 1, each product also has mean 0 and variance 1, and the variances of independent terms add up. So the score has variance d_k and a standard deviation of $\sqrt{d_k}$.

$$q \cdot k = \Sigma_i \, q_i k_i$$

Multiply the matching components of the query q and the key k, then add them up.

The figure draws one query and eight keys at random, with components from N(0, 1), and shows the softmax of their eight scores with no scaling at all. Drag d_k, the width of the vectors, from 1 to 512. The scores spread out roughly as $\sqrt{d_k}$, and a softmax that is soft at $d_k = 4$ turns into a single spike long before 512.

Wider vectors, sharper softmax, unless you divide by $\sqrt{d_k}$



score SD **2.02** top weight **0.57** entropy **1.85** bits

Shown at $d_k = 4$, divide by $\sqrt{d_k}$ off. Interactive version at explorable.geekplux.com/s/attention#fig-scaling

The query q and the eight keys are random, with N(0, 1) components (seed 229). Drag the width d_k; switch the $\sqrt{d_k}$ division on and off. Dashed outlines show the other setting.

4 The fix: divide by $\sqrt{d_k}$

Vaswani et al. suspect exactly this: for large d_k the dot products grow large in magnitude and push the softmax into regions where its gradients are extremely small. Their remedy is one division: every score is divided by $\sqrt{d_k}$ before the softmax. A score whose spread is $\sqrt{d_k}$, divided by $\sqrt{d_k}$, has a spread of about 1, whatever the width.

$$\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q stacks every word's query, K every key and V what each word passes on; each row of scores is divided by $\sqrt{d_k}$, softmaxed, and used to average V. Equation 1 of the paper.

Turn on “divide by $\sqrt{d_k}$” in the figure above and drag d_k again. The spread now stays near 1 and the weights stay soft at every width. In the paper's base model $d_k = 64$, so without the division the scores would be 8 times too spread out, the same as running the softmax at $T = 1/8$. Dividing by $\sqrt{d_k}$ is the temperature knob from the first figure, set to $T = \sqrt{d_k}$ so the scores always arrive at unit scale.

Why the square root and not d_k itself? Dividing by d_k would shrink the spread to $1/\sqrt{d_k}$, which is 0.04 at $d_k = 512$, and the weights would be nearly uniform: attention would see nothing. $\sqrt{d_k}$ is the one choice that keeps the spread fixed as the width changes.

Takeaway

The sharpness of a softmax depends on the scale of its scores, not just their order. Dot products of d_k-dimensional vectors spread out like $\sqrt{d_k}$, so attention divides by $\sqrt{d_k}$: a fixed temperature that hands the softmax unit-scale scores at any width.

Read the original paper